

COMPUTER PROGRAMMING

Electrical Engineering Department — UET Nowshera

LAB 4

Lab Title: Switch Statement, Break and Continue in C++

1. OBJECTIVE

The purpose of this lab is to develop a clear understanding of the switch statement in C++. Students also explore how the break statement manages execution flow within a switch block and how the continue statement functions inside loops. Additionally, this lab introduces the concept of building simple menu-driven programs using switch.

2. THEORY

The switch statement is employed when a program must choose among several possible options based on the value held by a variable. It is generally considered more readable and efficient than writing a long chain of if-else statements when a single variable is compared to a fixed set of constant values.

Within a switch block, each possible value is handled by a separate case label. The program evaluates the variable and jumps to the case that matches its value. The break statement is placed at the end of each case to stop execution and exit the switch; omitting it causes fall-through, where execution continues into the next case. The default label is used to handle any input that does not match any of the listed cases.

Inside a loop, the continue statement causes the current iteration to be skipped and immediately moves execution to the next iteration.

3. SOFTWARE USED

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

4. PROGRAMS

Program 1 — Day Name Using Switch

This program reads a number between 1 and 7 from the user and outputs the name of the corresponding day of the week using a switch statement.

Algorithm:

- Start the program.
- Declare an integer variable for the day number.
- Ask the user to enter a number from 1 to 7.
- Use a switch statement to match the number to a day name.
- Display the corresponding day name.
- End the program.

```
#include <iostream>
using namespace std;

int main()
{
    int day;

    cout << "Enter day number (1-7): ";
    cin >> day;

    // switch checks the value of 'day' against each case
    switch(day)
    {
        case 1: cout << "Monday";    break;
        case 2: cout << "Tuesday";   break;
        case 3: cout << "Wednesday"; break;
        case 4: cout << "Thursday";  break;
        case 5: cout << "Friday";    break;
        case 6: cout << "Saturday";  break;
        case 7: cout << "Sunday";    break;
        default: cout << "Invalid day number"; // runs if no case matches
    }

    return 0;
}
```

Program 2 — Grade Evaluation Using Switch

This program accepts a grade letter from the user and uses a switch statement to display the appropriate performance feedback.

Algorithm:

- Start the program.
- Declare a character variable for the grade.
- Ask the user to enter a grade (A, B, C, D, or F).
- Convert the grade to uppercase using toupper().
- Use a switch statement to evaluate the grade.
- Display the corresponding message.
- End the program.

```
#include <iostream>
using namespace std;

int main()
{
    char grade;

    cout << "Enter your grade (A, B, C, D, F): ";
    cin >> grade;
    grade = toupper(grade); // convert to uppercase for uniformity

    switch(grade)
    {
        case 'A': cout << "Excellent";      break;
        case 'B': cout << "Very Good";      break;
        case 'C': cout << "Good";           break;
        case 'D': cout << "Pass";           break;
        case 'F': cout << "Fail";           break;
        default:  cout << "Invalid Grade"; // handles unexpected input
    }

    return 0;
}
```

Program 3 — Unit Conversion Menu Using Switch

This program displays a menu of unit conversion options and carries out the conversion chosen by the user through a switch statement.

Algorithm:

- Start the program.
- Display a menu of conversion options.
- Take the user's menu choice.
- Ask the user to enter the value to convert.
- Perform the selected conversion using a switch statement.
- Display the result.
- End the program.

```
#include <iostream>
using namespace std;

int main()
{
    int choice;
    double value;

    // Display conversion menu
    cout << "1 -> Kilometers to Meters\n";
    cout << "2 -> Meters to Centimeters\n";
    cout << "3 -> Kilograms to Grams\n";
    cout << "4 -> Celsius to Fahrenheit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    cout << "Enter value: ";
    cin >> value;

    switch(choice)
    {
        case 1:
            cout << "Meters = " << value * 1000; break;
        case 2:
            cout << "Centimeters = " << value * 100; break;
        case 3:
            cout << "Grams = " << value * 1000; break;
        case 4:
            // Celsius to Fahrenheit formula: (C * 9/5) + 32
            cout << "Fahrenheit = " << (value * 9.0/5) + 32; break;
        default:
            cout << "Invalid choice";
    }

    return 0;
}
```

5. CONCLUSION

This lab demonstrated the switch statement as a cleaner and more efficient substitute for lengthy if-else chains, particularly in menu-based applications. The importance of the break statement in preventing unintended fall-through between cases was clearly illustrated, and the continue statement was introduced as a tool for skipping iterations within loops. A thorough understanding of these control flow structures is vital for developing interactive C++ programs.